

# 数据结构结课project实验报告

姓名：徐锦云

学号：23307130447

班级：张老师班

专业：信息安全

## 目录

### 数据结构结课project实验报告

目录

题目及题目分析

题目要求

题目分析

题目背景

问题核心

算法思路分析+代码

需要的函数

`canPass(int u, int v, int time_start, int time_end, int max_capacity)`

`addFlow(int u, int v, int time_start, int time_end)`

`dijkstra(int start, int end, std::vector<int> &finalPath, int &travelTime)`

Dijkstra 算法中的最小顶堆优先队列分析

1. 优先队列的作用
2. 最小顶堆优先队列的操作
3. 关键操作分析
4. 复杂度

`main()`

结果展示

小样例（体现代码逻辑）

大样例（主要看是否足够优化）

## 题目及题目分析

### 题目要求

## 二、Project 要求

### 2.1 基本要求

#### (1) 实现基础数据结构

- 设计记录城市交通网络的数据结构
- 设计记录车辆信息的数据结构

#### (2) 为多个车辆提供最短时间规划

给定多个车辆的起始地点和目的地点，系统需求得所有车辆从起点到终点的时间之和最短。车辆每单位时间可以行进的道路长度为 1。

### 2.2 高级要求

#### (1) 考虑道路车流量限制

在进行路径规划时，需要考虑道路的最大车流量。如果某条道路的车流量超过了其最大承载能力(例如，某条道路已经饱和，无法再容纳更多车辆)，则该道路将被排除。因此，需要系统在基础要求的基础上增加对道路车流量的考虑，以确保交通网络的流畅性。

## 三、实现细节

### 3.1 输入格式

输入共包含路径规划的基本信息、地点之间的道路长度、地点之间的交通流量限制、车辆的信息这几个部分。上述几个部分依次输入，以下是详细说明：

#### (1) 路径规划的基本信息

输入包含一行共 2 个正整数，分别为地点数量  $N$  ( $1 \leq N \leq 20$ ) 和车辆数量  $M$  ( $1 \leq M \leq 20$ )。

#### (2) 地点之间的道路长度

输入包含  $N$  行，每行包含  $N$  个用空格间隔开的数字。地点编号从 1 开始计算。输入的数字若为 0，则表示对应的两个地点之间没有道路连接；若为正整数，则表示对应的两个地点之间道路的长度。

### (3) 地点之间的交通流量限制

输入包含 N 行，每行包含 N 个用空格间隔开的数字。地点编号从 1 开始计算。其中 N 为地点的数量。输入的数字均为正整数，表示同一时间，最多能同时行驶车辆的数量。对于基本要求，此处输入的交通流量限制为足够大的正整数（即不需要考虑交通流量限制）。

### (4) 车辆的信息

输入包含 M 行，每行包含 2 个用空格间隔开的数字。M 行分别对应编号为 1 到 M 的车辆的信息。每行输入的第 1 个数字为车辆的出发地点编号，第 2 个数字为车辆的目的地编号。车辆每单位时间可以行进的道路长度为 1。

## 3.2 输出格式

输出共包含车辆的行驶路径和总的行驶时长两部分。

### (1) 车辆的行驶路径

输出 M 行，第 j 行对应编号为 j 的车辆的行驶路径信息。每一行包含车辆从其出发地开始到达目的地的完整路径。该路径使用 K 个数字表示，为该车辆经过的地点编号（包含出发地和目的地）。

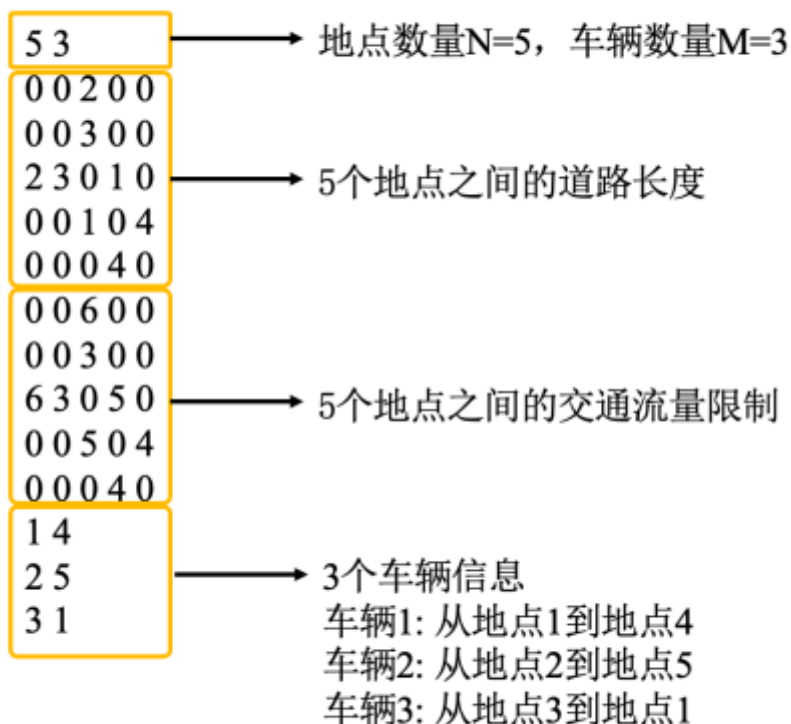
### (2) 总的行驶时长

输出 1 行，为所有车辆从出发地开始到目的地的总时长。

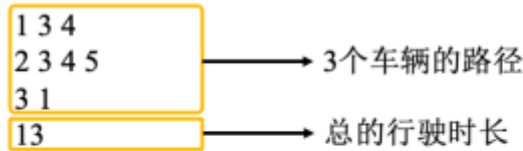
## 3.3 样例说明

### 测试样例

#### (1) 输入



#### (2) 输出



3个车辆的路径

车辆1: 车辆从地点1出发, 经过地点3, 到达地点4

车辆2: 车辆从地点2出发, 经过地点3和地点4, 到达地点5

车辆3: 车辆从地点3出发, 到达地点1

## 题目分析

### 题目背景

在一个交通路网中, 有若干个地点和道路, 每条道路都有特定的长度和流量限制。同时, 有多辆车需要从起点到达终点。要求设计一个算法, 为每辆车规划一条最短路径, 且在规划路径时必须考虑道路的流量限制。如果某条路段在某一时间段的交通流量已达到上限, 则车辆不能使用该路段, 而需要选择其他路径或等待。

### 问题核心

- 多源路径规划问题:** 需要为多个车辆规划从起点到终点的最短路径。
- 动态约束问题:** 规划路径时必须动态考虑道路流量的使用情况, 确保每条道路的流量不超过限制。
- 实时路径更新问题:** 路径规划不仅需要找到最短路径, 还需要更新道路的流量信息, 以保证后续车辆的规划结果正确。

## 算法思路分析+代码

一些量的定义:

```
#include <iostream>
#include <vector>
#include <limits>
#include <algorithm>
#include <queue>

#define INF std::numeric_limits<int>::max()

// 定义车辆结构体
struct Vehicle {
    int start, end;
    std::vector<int> path; // 路径
    int path_size;        // 路径长度

    Vehicle() : start(0), end(0), path_size(0) {}
};

// 定义流量区间结构体
struct FlowInterval {
    int interval_start;
    int interval_end;
    int used_capacity;
};
```

```
// 全局变量
int N, M; // 节点数和车辆数
std::vector<std::vector<int>>> dist;
std::vector<std::vector<int>>> capacity;
std::vector<std::vector<std::vector<FlowInterval>>>> flow;
std::vector<Vehicle> vehicles;
```

## 需要的函数

### canPass(int u, int v, int time\_start, int time\_end, int max\_capacity)

- **功能**: 检查车辆是否可以在指定时间段内通过从节点 `u` 到节点 `v` 的道路。道路上的流量不能超过该道路的最大容量。
- **思路**: 遍历存储该道路流量信息的 `flow[u][v]`，检查是否存在与当前车辆出发时间段重叠的流量区间。如果有，检查该时间段内流量是否超过道路容量。若超过，则返回 `false`；若没有冲突，则返回 `true`。
- **复杂度**: 该函数在最坏情况下遍历每条道路的所有流量区间，因此复杂度为  $O(K)$ ，其中  $K$  是该道路的流量区间数。

```
// 检查是否可以通过
bool canPass(int u, int v, int time_start, int time_end, int max_capacity) {
    for(auto &interval : flow[u][v]) {
        if(!(time_end <= interval.interval_start || time_start >=
interval.interval_end)) {
            if(interval.used_capacity >= max_capacity) {
                return false;
            }
        }
    }
    return true;
}
```

### addFlow(int u, int v, int time\_start, int time\_end)

- **功能**: 向流量信息中添加一条新的流量区间。
- **思路**: 为道路 `u` 到 `v` 添加一个新的流量区间，区间时间从 `time_start` 到 `time_end`，且流量为 `1`，表示该车通过。
- **复杂度**: 向 `flow[u][v]` 中添加一个元素，时间复杂度为  $O(1)$ 。

```
// 添加流量信息
void addFlow(int u, int v, int time_start, int time_end) {
    FlowInterval newInterval;
    newInterval.interval_start = time_start;
    newInterval.interval_end = time_end;
    newInterval.used_capacity = 1;
    flow[u][v].push_back(newInterval);
}
```

```
dijkstra(int start, int end, std::vector<int> &finalPath,
int &travelTime)
```

## Dijkstra 算法中的最小顶堆优先队列分析

### 1. 优先队列的作用

在 Dijkstra 算法中，我们需要从图的所有节点中找到当前距离源节点最近的未处理节点。为了能够高效地完成这一操作，我们使用优先队列。

- **最小顶堆** (Min-Heap) 是一种特殊的二叉堆结构，它的每一个父节点的值都小于或等于其子节点的值。因此，堆顶的节点（根节点）始终是当前队列中最小的节点。

Dijkstra 算法的具体过程是：

1. 初始化源节点的距离为 `0`，所有其他节点的距离为无穷大（`INF`）。
2. 将源节点加入优先队列，并开始从源节点扩展。
3. 每次从队列中弹出（即取出）当前距离源节点最小的节点 `u`，更新其相邻节点的最短路径。
4. 如果更新的路径更短，则将相邻节点更新到优先队列中。
5. 重复此过程，直到所有节点的最短路径都被确定。

### 2. 最小顶堆优先队列的操作

在 Dijkstra 算法中，最小顶堆优先队列需要支持以下基本操作：

- **插入操作**：将一个新的节点和其对应的距离插入队列中。
- **删除最小元素**：从堆顶删除当前最小的节点（即当前距离源节点最近的节点）。
- **更新操作**：更新某个已存在节点的距离，并根据新的距离重新调整堆的结构。

### 3. 关键操作分析

插入操作：

在 Dijkstra 算法中，每次扩展一个节点时，可能会将新节点加入到优先队列中。我们用 `pq.push()` 将一个新的节点插入队列，其中该节点存储的信息是 `(distance, node)`，即节点的距离和节点的编号。

- **时间复杂度**：插入操作的时间复杂度为  $O(\log N)$ ，其中 `N` 是队列中的元素数量。因为插入操作需要将新元素插入堆并维护堆的性质。

删除最小元素：

在每一步的 Dijkstra 算法中，我们要从队列中取出最小距离的节点，这个操作通常是用 `pq.top()` 查看堆顶元素，然后用 `pq.pop()` 将其删除。此时，堆会自动重新调整，以保证堆顶始终是当前最小的元素。

- **时间复杂度**：删除堆顶元素的时间复杂度为  $O(\log N)$ ，因为删除堆顶元素后，堆会重新进行调整，以保持堆的有序性。

更新操作（减少键值）：

在 Dijkstra 算法中，一旦通过某个节点 `u` 到达一个相邻节点 `v`，我们需要检查是否能够通过更新 `v` 的距离来得到更短的路径。如果是，则需要在堆中更新 `v` 的距离。这时我们不能直接修改堆中的值，而是必须删除原有元素并插入一个新元素。

## 4. 复杂度

- 时间复杂度主要由 Dijkstra 算法的计算复杂度决定： $O((N + E) * \log N)$ ，其中  $N$  为节点数， $E$  为边数。
- 由于每次扩展时还需要检查 `canPass` 函数的流量限制，最坏情况下复杂度为  $O(K)$ ，其中  $K$  为流量区间数。因此，整体的时间复杂度是  $O((N + E) * \log N * K)$ 。

```
// 使用小顶堆优先队列按最早到达时间进行扩展
bool dijkstra(int start, int end, std::vector<int> &finalPath, int &travelTime) {
    static bool earliest[700][300];
    static int parent[700][300];
    static int parentTime[700][300];

    for(int i = 0; i < N; i++){
        for(int t = 0; t < 300; t++){
            earliest[i][t] = false;
            parent[i][t] = -1;
            parentTime[i][t] = -1;
        }
    }

    // 小顶堆，元素为 (time, node)
    std::priority_queue<
        std::pair<int, int>,
        std::vector<std::pair<int, int>>,
        std::greater<std::pair<int, int>>
    > pq;

    earliest[start][0] = true;
    parent[start][0] = start;
    parentTime[start][0] = 0;
    pq.push({0, start});

    int foundTime = -1;
    while(!pq.empty()){
        auto [curT, u] = pq.top();
        pq.pop();
        if(u == end){
            foundTime = curT;
            break;
        }
        // 再次判断是否与 earliest[] 状态一致(可能已被更新)
        if(!earliest[u][curT]) continue;

        // 扩展到相邻节点
        for(int v = 0; v < N; v++){
            int cost = dist[u][v];
            if(cost <= 0) continue;
            int arrivalT = curT + cost;
            if(arrivalT < 300 && !earliest[v][arrivalT]) {
                // 检查容量限制
                if(canPass(u, v, curT, arrivalT, capacity[u][v])) {
                    earliest[v][arrivalT] = true;
                    parent[v][arrivalT] = u;
                    parentTime[v][arrivalT] = curT;
                    pq.push({arrivalT, v});
                }
            }
        }
    }
}
```



```

    }
}

if(foundTime < 0) return false; // 无法到达
travelTime = foundTime;

// 回溯
std::vector<int> path;
int curNode = end, curT = foundTime;
while(!(curNode == start && curT == 0)) {
    path.push_back(curNode);
    int pn = parent[curNode][curT];
    int pt = parentTime[curNode][curT];
    curNode = pn;
    curT = pt;
}
path.push_back(start);
std::reverse(path.begin(), path.end());
finalPath = path;
return true;
}

```

## main()

- **功能：**程序的主入口，负责读取输入、初始化数据、调用算法并输出结果。
- **思路：**
  1. 读取输入数据，包括图的结构（`dist` 和 `capacity`）、车辆的起点和终点。
  2. 使用 Dijkstra 算法为每辆车计算最短路径，并在路径上添加流量信息。
  3. 输出每辆车的路径和所有车辆的总时间。
- **复杂度：**
  - 初始化阶段（读取输入）是  $O(N^2 + M)$ ，其中  $N^2$  是初始化矩阵和流量数据， $M$  是车辆数。
  - 对于每辆车，调用 `dijkstra` 算法，因此整体时间复杂度为  $O(M * ((N + E) * \log N * K))$ 。

```

int main() {
    std::cin >> N >> M;
    // 初始化距离和容量矩阵
    dist.assign(N, std::vector<int>(N, 0));
    capacity.assign(N, std::vector<int>(N, 0));
    flow.assign(N, std::vector<std::vector<FlowInterval>>(N,
std::vector<FlowInterval>()));
    // 距离矩阵
    for(int i = 0; i < N; ++i)
        for(int j = 0; j < N; ++j)
            std::cin >> dist[i][j];
    // 容量矩阵
    for(int i = 0; i < N; ++i)
        for(int j = 0; j < N; ++j)
            std::cin >> capacity[i][j];
    // 读取车辆起点和终点
    vehicles.resize(M);
    for(int i = 0; i < M; ++i) {
        std::cin >> vehicles[i].start >> vehicles[i].end;
    }
}

```



```

        vehicles[i].start--;
        vehicles[i].end--;
    }
    int totalTime = 0;
    for(int i = 0; i < M; ++i) {
        std::vector<int> path;
        int travelTime = 0;
        // 新 dijkstra
        if(dijkstra(vehicles[i].start, vehicles[i].end, path, travelTime)) {
            // 将搜索到的路径写入车辆
            int time_start = 0;
            for(int j = 1; j < (int)path.size(); ++j) {
                int u = path[j-1];
                int v = path[j];
                int time_end = time_start + dist[u][v];
                addFlow(u, v, time_start, time_end);
                time_start = time_end;
            }
            vehicles[i].path = path;
            vehicles[i].path_size = (int)path.size();
            totalTime += travelTime;
        }
    }
    for(auto &vehicle : vehicles) {
        for(auto node : vehicle.path)
            std::cout << (node + 1) << " ";
        std::cout << "\n";
    }
    std::cout << totalTime << "\n";
    return 0;
}

```

## 结果展示

跑出来结果如下：

### 小样例（体现代码逻辑）

第一种基础情况，流量没限制：正确

```

5 3
0 0 2 0 0
0 0 3 0 0
2 3 0 1 0
0 0 1 0 4
0 0 0 4 0
0 0 6 0 0
0 0 3 0 0
6 3 0 5 0
0 0 5 0 4
0 0 0 4 0
1 4
2 5
3 1
1 3 4
2 3 4 5
3 1
13

```

第二种情况，流量有限制，第一辆车选择最短路，第二辆车被迫选择次短路：正确

```

● PS C:\Code> g++ -o test1 test1.cpp
● PS C:\Code> ./test1
5 3
0 0 3 0 0
0 0 3 5 0
3 3 0 1 0
0 5 1 0 4
0 0 0 4 0
0 0 2 0 0
0 0 1 1 0
2 1 0 1 0
0 1 1 0 1
0 0 0 1 0
1 4
2 5
3 1
1 3 4
2 4 5
3 1
16

```

第三种情况，流量有限制，但是时间上恰好错开了，基于对时间窗口的判断，仍然可以走最短路：正确

```
PS C:\Code> g++ -o test1 test1.cpp
PS C:\Code> ./test1
5 3
0 0 3 0 0
0 0 2 5 0
3 2 0 1 0
0 5 1 0 4
0 0 0 4 0
0 0 2 0 0
0 0 1 1 0
2 1 0 1 0
0 1 1 0 1
0 0 0 1 0
1 4
2 5
3 1
1 3 4
2 3 4 5
3 1
14
```

第四种情况，车辆被迫需要掉头：正确

```
PS C:\Code> g++ -o test1 test1.cpp
PS C:\Code> ./test1
5 2
0 0 1 0 0
0 0 1 0 0
1 1 0 1 1
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0
1 1 0 1 1
0 0 1 0 0
0 0 1 0 0
1 4
2 4
1 3 4
2 3 1 3 4
6
```

第五种情况，车辆可以确定走不了最短路，但是可以选择到底是掉头还是回溯，在这其中选择了较短者：正确

```
PS C:\Code> g++ -o test1 test1.cpp
PS C:\Code> ./test1
7 2
0 0 1 0 0 0 0
0 0 1 0 1 0 0
1 1 0 1 0 0 0
0 0 1 0 0 0 1
0 1 0 0 0 1 0
0 0 0 0 1 0 1
0 0 0 1 0 1 0
0 0 1 0 0 0 0
0 0 1 0 1 0 0
1 1 0 1 0 0 0
0 0 1 0 0 0 1
0 1 0 0 0 1 0
0 0 0 0 1 0 1
0 0 0 1 0 1 0
1 4
2 4
1 3 4
2 3 1 3 4
6
```

## 大样例（主要看是否足够优化）

大数据量检验：已经趋近最优解

第一组：

```
PS C:\Code> g++ -o test1 test1.cpp
```

```
PS C:\Code> ./test1
```

● 20 20

```
0 0 70 40 75 0 0 0 0 0 0 55 0 92 17 0 0 68 0 0
0 0 96 5 0 0 0 0 9 0 0 60 0 0 0 0 0 0 62
70 96 0 59 0 0 82 0 0 0 0 29 0 13 69 0 0 0 0 0
40 5 59 0 0 0 48 66 0 0 0 0 0 0 1 33 0 0 0
75 0 0 0 0 0 10 87 0 0 0 0 0 0 44 74 0 52 0
0 0 0 0 0 0 0 16 0 32 0 0 0 0 0 0 0 0 0
0 0 82 48 10 0 0 0 0 0 0 0 0 100 0 1 0 0 0
0 0 0 66 87 0 0 0 1 59 0 0 0 0 76 41 0 0 0 0
0 9 0 0 0 16 0 1 15 0 0 0 0 0 0 0 31 0 6
0 0 0 0 0 0 59 0 0 0 0 0 0 27 0 0 5 21 0
0 0 0 0 0 32 0 0 0 0 0 0 0 0 0 0 0 79 90
55 60 29 0 0 0 0 0 0 0 0 77 0 0 27 43 0 89 78 0
0 0 0 0 0 0 0 0 0 0 0 16 0 0 0 0 0 0 0
92 0 13 0 0 0 0 0 0 0 0 16 0 0 0 0 9 0 32
17 0 69 0 0 0 100 76 0 27 0 27 0 0 0 0 0 0 18 0
0 0 0 1 44 0 0 41 0 0 0 43 0 0 0 15 0 0 0 0
0 0 0 33 74 0 1 0 0 0 0 0 0 0 0 0 0 80 44
68 0 0 0 0 0 0 31 5 0 89 0 9 0 0 0 84 0 0
0 0 0 0 52 0 0 0 21 79 78 0 0 18 0 80 0 15 0
0 62 0 0 0 0 0 6 0 90 0 0 32 0 0 44 0 0 52
0 0 3 1 3 0 0 0 0 0 1 0 1 2 0 0 3 0 0
0 0 2 3 0 0 0 0 2 0 0 1 0 0 0 0 0 0 1
3 2 0 3 0 0 2 0 0 0 0 3 0 1 1 0 0 0 0 0
1 3 3 0 0 0 3 2 0 0 0 0 0 0 2 3 0 0 0
3 0 0 0 0 0 2 3 0 0 0 0 0 0 1 2 0 1 0
0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0
0 0 2 3 2 0 0 0 0 0 0 0 0 3 0 3 0 0 0
0 0 0 2 3 0 0 0 3 3 0 0 0 0 3 2 0 0 0 0
0 2 0 0 0 1 0 3 3 0 0 0 0 0 0 0 2 0 1
0 0 0 0 0 0 3 0 0 0 0 0 0 1 0 0 2 3 0
0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 2
1 1 3 0 0 0 0 0 0 0 1 0 0 2 3 0 1 1 0
0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
1 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 1
```

```
2 0 1 0 0 0 3 3 0 1 0 2 0 0 0 0 0 0 2 0
0 0 0 2 1 0 0 2 0 0 0 3 0 0 0 1 0 0 0 0
0 0 0 3 2 0 3 0 0 0 0 0 0 0 0 0 0 0 2 1
3 0 0 0 0 0 0 0 2 2 0 1 0 1 0 0 0 3 0 0
0 0 0 0 1 0 0 0 0 3 1 1 0 0 2 0 2 0 3 0
0 1 0 0 0 0 0 0 1 0 2 0 0 1 0 0 1 0 0 2
1 6
1 7
1 13
1 15
1 20
2 9
2 18
2 20
3 4
4 7
5 14
5 17
6 7
6 13
6 19
8 12
11 13
12 15
12 16
13 14
```

输出：

```
1 4 2 9 6
1 5 7
1 15 10 18 14 13
1 15
1 15 19 10 18 9 20
2 9
2 9 18
2 9 20
3 4
4 17 7
5 7 17 20 14
5 7 17
6 9 2 4 17 7
6 11 6 9 8 9 20 14 13

8 9 2 4 16 12
11 6 9 18 14 13
12 15
12 16
13 14
1048
```

第二组：



```
● PS C:\Code> g++ -o test1 test1.cpp
● PS C:\Code> ./test1
20 20
46 25 66 57 0 0 0 0 63 0 29 0 0 3 0 0 0 0 36 98
25 0 29 55 39 0 58 0 77 0 0 0 0 0 0 0 0 15 0 0
66 29 0 14 89 84 0 0 22 27 0 0 0 0 0 0 0 0 96
57 55 14 0 0 0 39 24 0 20 38 52 0 88 0 0 61 0 0 15
0 39 89 0 0 0 0 0 0 0 90 67 0 0 8 47 70 0 0
0 0 84 0 0 0 0 0 68 0 27 0 0 0 6 0 72 41 100 81
0 58 0 39 0 0 0 0 43 0 0 0 0 0 24 36 0 0 0 0
0 0 0 24 0 0 0 0 2 0 0 0 0 0 0 0 96 0 27
63 77 22 0 0 68 43 2 0 0 0 0 62 0 0 0 37 0 35 0
0 0 27 20 0 0 0 0 0 0 0 76 0 0 0 48 0 0 26 0
29 0 0 38 0 27 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 52 90 0 0 0 0 76 0 0 0 0 0 0 41 0 0 0
0 0 0 0 67 0 0 0 62 0 0 0 59 28 0 0 0 40 0 0
3 0 0 88 0 0 0 0 0 0 0 28 0 0 0 0 0 0 51
0 0 0 0 0 6 24 0 0 0 0 0 0 0 0 0 0 0 48
0 0 0 0 8 0 36 0 0 48 0 0 0 0 0 0 0 0 0
0 0 0 61 47 72 0 0 37 0 0 41 0 0 0 0 51 0 0 0
0 15 0 0 70 41 0 96 0 0 0 0 40 0 0 0 0 0 0
36 0 0 0 0 100 0 0 35 26 0 0 0 0 0 0 0 0 0
98 0 96 15 0 81 0 27 0 0 0 0 0 51 48 0 0 0 0 82
1 1 1 1 0 0 0 0 1 0 1 0 0 1 0 0 0 0 1 1
1 0 1 1 1 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0
1 1 0 1 1 1 0 0 1 1 0 0 0 0 0 0 0 0 1
1 1 1 0 0 0 1 1 0 1 1 1 0 1 0 0 1 0 0 1
0 1 1 0 0 0 0 0 0 0 0 1 1 0 0 1 1 1 0 0
0 0 1 0 0 0 0 0 1 0 1 0 0 0 1 0 1 1 1 1
0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 1 0 0 0 0
0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 1 0 1
1 1 1 0 0 1 1 1 0 0 0 0 1 0 0 0 1 0 1 0
0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 1 0 0 1 0
1 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 1 1 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0
0 0 0 0 1 0 0 0 1 0 0 0 1 1 0 0 0 1 0 0
1 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1
```

```
0 0 0 1 1 0 0 0 0 1 0 0 0 0 0 1 0 0 0
0 0 0 0 1 0 0 0 1 0 0 0 1 1 0 0 0 1 0 0
1 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1
0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1
0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0
0 0 0 1 1 1 0 0 1 0 0 1 0 0 0 0 1 0 0 0
0 1 0 0 1 1 0 1 0 0 0 0 1 0 0 0 0 0 0
1 0 0 0 0 1 0 0 1 1 0 0 0 0 0 0 0 0 0
1 0 1 1 0 1 0 1 0 0 0 0 0 1 1 0 0 0 1
1 11
2 6
2 8
2 12
3 4
3 19
4 11
5 8
5 14
6 7
7 10
7 20
10 13
11 13
11 18
12 20
13 20
14 15
14 20
15 16
```

输出：

```
1 11
2 18 6
2 3 9 8
2 4 12
3 4
3 10 19
4 11
5 17 9 8
5 2 1 14
6 15 7
7 4 10
7 15 20
10 4 8 9 13
11 1 14 13
11 6 18
12 4 20
13 14 20
14 1 14 1 14 1 14 1 2 18 6 15
14 4 20
15 6 15 6 15 6 15 7 16
1353
```

第三组：

```
PS C:\Code> g++ -o test1 test1.cpp
```

```
PS C:\Code> ./test1
```

● 20 20

```
0 0 0 0 0 0 88 0 45 68 0 0 30 0 0 0 0 0 0
0 18 0 0 30 0 0 16 0 0 0 0 0 0 0 23 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 27 0 0 53 0
0 0 0 0 0 33 0 0 0 0 0 0 32 0 91 50 0 0 36
0 30 0 0 0 0 0 0 0 0 34 0 19 0 10 11 0 0 0
0 0 0 33 0 0 3 0 0 0 4 88 0 0 0 0 0 0 0
88 0 0 0 0 3 0 0 0 0 0 0 0 35 47 0 0 0 23
0 16 0 0 0 0 0 0 51 0 82 0 0 4 0 0 0 0 0
45 0 0 0 0 0 0 51 0 0 0 0 0 50 0 0 0 0 85 0
68 0 0 0 0 0 0 0 0 0 74 46 0 0 0 0 0 0 0
0 0 0 0 34 4 0 82 0 74 0 91 0 0 0 0 0 0 81 0
0 0 0 0 0 88 0 0 0 46 91 90 26 0 0 48 0 0 0 16
30 0 0 0 19 0 0 0 0 0 26 19 0 0 93 0 88 99 0
0 0 0 32 0 0 0 4 50 0 0 0 0 0 0 0 0 0 31
0 0 0 0 10 0 35 0 0 0 0 0 0 0 0 0 0 0 0
0 0 27 91 11 0 47 0 0 0 0 48 93 0 0 0 61 0 0 0
0 23 0 50 0 0 0 0 0 0 0 0 0 0 61 53 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 88 0 0 0 0 38 0 0
0 0 53 0 0 0 0 0 85 0 81 0 99 0 0 0 0 0 0 0
0 0 0 36 0 0 23 0 0 0 0 16 0 31 0 0 0 0 0 0
0 0 0 0 0 0 1 0 1 1 0 0 1 0 0 0 0 0 0 0
0 1 0 0 1 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0
0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 1 1 0 0 1
0 1 0 0 0 0 0 0 0 0 1 0 1 0 1 1 0 0 0 0
0 0 0 1 0 0 1 0 0 0 1 1 0 0 0 0 0 0 0 0
1 0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 0 0 0 1
0 1 0 0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0
1 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 1 0
1 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0
0 0 0 0 1 1 0 1 0 1 0 1 0 0 0 0 0 0 1 0
0 0 0 0 0 1 0 0 0 1 1 1 1 0 0 1 0 0 0 1
1 0 0 0 1 0 0 0 0 0 0 1 1 0 0 1 0 1 1 0
0 0 0 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 1
```

```
1 0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 0 0 0 1
0 1 0 0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0
1 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 1 0
1 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0
0 0 0 0 1 1 0 1 0 1 0 1 0 0 0 0 0 0 1 0
0 0 0 0 0 1 0 0 0 0 1 1 1 1 0 0 1 0 0 0 1
1 0 0 0 1 0 0 0 0 0 0 0 1 1 0 0 1 0 1 1 0
0 0 0 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1
0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 1 1 1 0 1 0 0 0 0 0 1 1 0 0 0 1 0 0 0
0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0 1 0 1 0 1 0 0 0 0 0 0 0
0 0 0 1 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 0
1 9
1 14
1 16
1 19
2 11
2 13
2 20
3 10
3 17
4 8
4 14
4 19
5 12
6 14
7 20
9 19
9 20
14 17
15 17
17 18
```

输出：

```
1 9
1 13 5 2 8 14
1 7 16
1 10 11 19
2 5 11
2 8 2 5 13
2 2 8 14 20
3 16 12 10
3 19 3 16 17
4 14 8
4 20 14
4 6 11 19
5 13 12
6 4 14
7 20
9 19
9 8 14 8 14 8 14 20
14 4 17
15 5 2 17
17 16 5 15 5 13 18
1916
```